

Module 3 — Class 5 Assignment: Pipelines

Type-it-yourself exercise

UN AI/ML Mentorship

Module 3 — Class 5 Assignment: Pipelines

Topic: Pipelines — Pipeline + ColumnTransformer with a Random Forest

Goal: Open a **fresh Google Colab notebook** and type the code yourself by following the steps below. Do NOT copy-paste from the working notebook. Typing it yourself is how you learn the syntax.

How to submit

1. Open Google Colab → File → New Notebook.
 2. Type the code from each step below. Add a short # comment on EVERY cell so we know you understand it.
 3. Save the notebook as `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`.
 4. Push it to your team repo at `module-<X>/class_<Y>/submissions/<YourName>/.`
-

Step 0 — Load and prepare

```
import pandas as pd
url = 'https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv'
df = pd.read_csv(url)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce').fillna(0)
df['Churn_bin'] = df['Churn'].map({'No': 0, 'Yes': 1})
```

Step 1 — Pick features and split into train/test

What to do: Use DIFFERENT features than the working notebook.

Type this in a new cell:

```
num_features = ['tenure', 'MonthlyCharges', 'SeniorCitizen']
cat_features = ['Contract', 'InternetService', 'PaymentMethod']
X = df[num_features + cat_features]
y = df['Churn_bin']
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
print(f'Train: {X_train.shape}, Test: {X_test.shape}')
```

Expected output:

Train: (5282, 6), Test: (1761, 6)

Step 2 — Numeric pipeline (imputer + MinMaxScaler)

What to do: Use MinMax this time (not Standard).

Type this in a new cell:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', MinMaxScaler())
])
```

Expected output:

(no output, the pipeline is just defined)

Step 3 — Categorical pipeline (imputer + OneHotEncoder)

What to do: Use most_frequent for missing categoricals.

Type this in a new cell:

```
from sklearn.preprocessing import OneHotEncoder
cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])
```

Expected output:

(no output)

Step 4 — Combine with ColumnTransformer

What to do: Apply num_pipe to numeric cols, cat_pipe to categorical cols.

Type this in a new cell:

```
from sklearn.compose import ColumnTransformer
preprocessor = ColumnTransformer([
    ('num', num_pipe, num_features),
    ('cat', cat_pipe, cat_features)
])
```

Expected output:

(no output)

Step 5 — Full pipeline with RandomForestClassifier

What to do: Different model than the working notebook (which used LogisticRegression).

Type this in a new cell:

```
from sklearn.ensemble import RandomForestClassifier
full_pipe = Pipeline([
    ('preprocess', preprocessor),
    ('model', RandomForestClassifier(n_estimators=100, random_state=42))
])
full_pipe
```

Expected output:

Pipeline displayed with preprocess + model steps.

Step 6 — Train the whole pipeline in one line

What to do: fit() runs preprocess + train together.

Type this in a new cell:

```
full_pipe.fit(X_train, y_train)
print('Trained.')
```

Expected output:

Trained.

Step 7 — Evaluate test accuracy

What to do: Use score() directly on the pipeline.

Type this in a new cell:

```
acc = full_pipe.score(X_test, y_test)
print(f'Random Forest test accuracy: {acc:.4f}')
```

Expected output:

Random Forest test accuracy: around 0.78-0.80

Step 8 — Save and reload the pipeline

What to do: Use joblib.

Type this in a new cell:

```
import joblib
joblib.dump(full_pipe, 'rf_pipeline.joblib')
loaded = joblib.load('rf_pipeline.joblib')
print('Loaded model test accuracy:', round(loaded.score(X_test, y_test), 4))
```

Expected output:

Loaded model test accuracy: same as above.

Checklist before you submit

- ☐ All cells run without error
- ☐ Every code cell has a # comment in your own words
- ☐ Outputs are visible (you saved AFTER running)
- ☐ File name is correct: `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`

Deadline: before next class.